

Settings and context()

Intent

`dspy.settings` is the singleton that holds the language model, the adapter, the retriever, the callbacks, and a handful of runtime flags every other component reads. You configure it once at startup with `dspy.configure(...)` and override it locally with `with dspy.context(...)`. This page covers what's in there, how the global and scoped layers interact, and how overrides do (and don't) propagate across threads and async tasks.

Read this when you need to swap LMs mid-program, run different branches under different adapters, debug why a parallel worker isn't seeing your context override, or write a new module that needs to read from settings.

Design decisions

1. Two entry points by design

`configure` is for process-wide, one-time setup; `context` is for scoped overrides. They behave differently and have different ownership rules. `configure` mutates the global dict; `context` pushes onto a `ContextVar`. Splitting them makes the intent of each call clear, and lets the framework enforce one-time setup for the global while allowing freely-nested overrides for the scoped one.

2. Two layers underneath

A global `main_thread_config` dict and a `thread_local_overrides` `ContextVar`. Reads check the override first, then the global. A `dspy.context(lm=alt_lm)` block doesn't touch the global config — it shadows it for the duration of the block, on the current execution context. Every read goes through the same merge, which is why an adapter swap inside a context block is honored even by sub-modules constructed before the block opened.

3. configure has an owner thread

The first thread to call it owns it; calls from other threads raise `RuntimeError`. The intent is to make `configure` look like Python module-level initialization. If multiple threads were allowed to reconfigure mid-program, you'd get races where one thread's `lm` swap silently appeared in another thread's prediction. IPython is exempt — interactive sessions need the flexibility.

4. context uses Python's contextvars

Overrides propagate correctly across `await` and `asyncio.create_task`. `threading.local()` would have lost the override at the first `await`. `contextvars` ties the override to the execution context, not the thread, and the standard library copies it automatically to tasks spawned within. That's what makes `with dspy.context(lm=alt): await my_async_module(...)` work the way you'd hope.

5. DSPy's executors snapshot and re-apply

`dspy.Parallel`, `Module.batch`, and `asyncify` capture the parent's overrides and re-set them inside each worker thread. Plain `ContextVar` inheritance doesn't survive crossing into a new OS thread — Python only auto-propagates across `asyncio` task boundaries. So the executors do it themselves: grab `thread_local_overrides.get()` at submission, `.set()` it inside the worker before running user code, and reset on the way out. `usage_tracker` is deep-copied per worker so each thread accounts independently.

6. Plain Python threads do not inherit overrides

`threading.Thread` and bare `concurrent.futures.ThreadPoolExecutor` won't see your `with dspy.context(...)` block. This trips people up. If you spin up your own thread pool and run DSPy modules in the workers, the context override stays in the parent. The fix: capture `dspy.settings.thread_local_overrides.get()` in the parent and re-set it inside each worker, or use `dspy.Parallel`, which does it for you.

7. Settings are read lazily at call sites

Predict reads `settings.lm` at call time; Adapter reads `settings.adapter`; Module reads `settings.track_usage`. No constructor takes a `settings` argument. The cost is implicit dependency — a module's behavior depends on whatever context surrounds the call. The benefit is composition: settings overrides flow into every sub-module without rewiring.

8. Some knobs live on dspy.LM, not settings

`temperature`, `max_tokens`, `api_base`, `retries`. The split is intentional. Settings is for orchestration knobs that apply across LMs — which LM is the default, what adapter to use, whether usage is being tracked. LM-instance knobs (sampling parameters, endpoint, retry policy) live on the LM itself, because a program might use several different LMs with different settings.

9. Locking is minimal

A single `Lock` guards only `_ensure_configure_allowed`. Reads are unsynchronized. The lock serializes ownership checks during `configure`; the read path doesn't take it. CPython dict-item access is atomic enough for the read pattern, and adding locking would have hurt every Predict call. `configure` is meant to be called once at startup; once it has settled, reads are uncontended.

API walkthrough

Grouped by what you're trying to do.

Setting things up

dspy.configure(kwargs)**
Merges `kwargs` into the global `main_thread_config` dict. Thread-safe via owner-thread enforcement: calls from other threads raise `RuntimeError`. Same restriction applies across async tasks, with the IPython exception. Use it once at startup; for a mid-program swap, prefer `context`.

dspy.context(kwargs)**
Context manager. On entry, pushes `kwargs` onto the `thread_local_overrides` `ContextVar`; on exit, resets the `ContextVar` token. Safe to call from any thread or async task; nests cleanly. Use it inside parallel branches, inside `asyncify`-wrapped functions, anywhere you need a scoped override.

dspy.settings
The singleton. Read attributes directly: `dspy.settings.lm`, `dspy.settings.adapter`. Reads merge global + override on every access — no caching, so a context-block change is visible immediately to every sub-module that reads.

dspy.settings.copy() / dspy.settings.config
Return a merged dotdict of the current effective config. Useful when you want to snapshot settings — to pass to a worker, to log, to compare before/after.

dspy.load_settings(path, allow_pickle=False) / dspy.settings.save(path, modules_to_serialize=None, exclude_keys=None)
Round-trip the settings dict to disk via cloudpickle. `exclude_keys` lets you drop sensitive fields (API keys aren't in settings, but custom keys you've added might be).

What you can configure

Standard keys (defined in `DEFAULT_CONFIG`):

Key	Default	Purpose
<code>lm</code>	<code>None</code>	Default LM for every <code>Predict</code> .
<code>adapter</code>	<code>None</code> (falls back to <code>ChatAdapter()</code>)	Adapter used to format and parse.
<code>rm</code>	<code>None</code>	Default retriever for <code>dspy.Retrieve()</code> .
<code>callbacks</code>	<code>[]</code>	Callback handlers fired around module calls.
<code>track_usage</code>	<code>False</code>	Enable token-usage accounting.
<code>num_threads</code>	<code>8</code>	Default thread count for <code>dspy.Parallel</code> / <code>Module.batch</code> .
<code>async_max_workers</code>	<code>8</code>	Concurrency cap for <code>asyncify</code> .
<code>max_errors</code>	<code>10</code>	Parallel-job error threshold before cancellation.
<code>provide_traceback</code>	<code>False</code>	Include tracebacks in parallel error logs.
<code>disable_history</code>	<code>False</code>	Skip recording call history on modules.
<code>warn_on_type_mismatch</code>	<code>True</code>	Warn when an input doesn't match the signature's declared type.
<code>max_history_size</code> / <code>max_trace_size</code>	<code>10000</code> each	Trim limits for module history and per-call trace.
<code>stream_listeners</code> / <code>send_stream</code>	<code>[]</code> / <code>None</code>	Streaming wiring.
<code>allow_tool_async_sync_con</code> / <code>version</code>	<code>False</code>	Permit running async tools inside sync code.
<code>branch_idx</code>	<code>0</code>	Rollout / branch index used by some optimizers.

Set-by-DSPy keys (don't set these yourself):

- `caller_predict`, `caller_modules` — the current call stack, used by callbacks and tracing.
- `usage_tracker`, `trace` — populated by the relevant contexts and read inside modules.

How overrides propagate

The mechanics that decide which threads and tasks see an override.

ContextVar propagation

`dspy.context(...)` sets a `contextvars.ContextVar`. Python copies the active context automatically to `asyncio.create_task`, `asyncio.gather`, and inherits it across `await`. No DSPy code needed for this — it's how `contextvars` works.

dspy.Parallel / Module.batch

At submission time, captures the parent's `thread_local_overrides.get()` and hands the dict to each worker. The worker calls `thread_local_overrides.set(parent_overrides)` before running user code and resets on the way out. `usage_tracker` is deep-copied per worker so each thread accounts independently rather than racing on a shared object.

dspy.asyncify(fn)

Runs a sync DSPy module on a worker thread via `anyio.to_thread.run_sync`. Same snapshot-and-reapply trick: capture parent overrides at call time, re-apply in the worker, reset on exit.

Plain threading.Thread / concurrent.futures.ThreadPoolExecutor

Does not inherit. The override lives in a `ContextVar` that the new OS thread doesn't see. Workaround: capture `dspy.settings.thread_local_overrides.get()` in the parent and `set` it inside the worker, or wrap the work in `dspy.Parallel`.

Ownership and locking

Owner thread

First call to `dspy.configure` records `config_owner_thread_id`. Later calls from a different thread (or, in async, a different task) raise `RuntimeError`. Multiple `configure` calls from the owner are allowed and overwrite in place.

IPython exception

Multiple `configure` calls from different async tasks in the same IPython session are allowed, detected via `get_ipython()`. The reason: notebook cells often run in fresh task contexts, and forcing each to be single-shot would break interactive use.

The lock

A single `threading.Lock` guards `_ensure_configure_allowed`. The read path doesn't take it; mutations rely on dict-item assignment being atomic in CPython. Reads inside `Predict` and `Adapter` are uncontended, which matters because they happen on every call.

Cross-links

- [Saving and loading](#) — settings serialization sits next to module serialization.
- Reference: `clients.md` — LM-instance knobs (`temperature`, `max_tokens`, `api_base`, `retries`) that intentionally don't live in settings.